

## 4.1 Tool Description

Valgrind Memcheck is a dynamic analysis tool that instruments a program's memory operations at runtime to detect memory errors and leaks. It belongs to the dynamic analysis category, executing the program under a synthetic CPU that tracks every memory allocation, deallocation, and access. Compared to the static analyzers used in Exercise 1 (Infer and clang-analyzer), Valgrind operates on compiled binaries rather than source code. This means it produces zero false positives on executed paths, since every error it reports is a real bug, but it is fundamentally limited to paths that are actually executed. Bugs in unexecuted branches are invisible to it, whereas static analyzers can reason about all possible paths. Compared to AFL++ (Exercise 2), Valgrind does not generate or mutate inputs. It observes a single execution with extreme precision rather than exploring many executions with random inputs. Combining both, by running AFL++-generated inputs under Valgrind, is a common practice that gets the best of both worlds. Compared to KLEE (Exercise 3), Valgrind performs no symbolic reasoning and explores no alternative paths. It is best suited for finding memory leaks, use-after-free errors, buffer overflows, and invalid memory accesses in well-exercised code paths.

## 4.2 Tool Application

Since Valgrind is a dynamic analysis tool, it only observes code paths that are actually executed. The original `hashmap.c` `main()` exercises only a single happy-path scenario, leaving most bug-triggering paths unexecuted. To expose a broader set of bugs, an extended test driver (`hashmap_test.c`) was created that exercises key update, long key insertion, and key deletion in addition to the basic `set/get` flow.

### Screenshot 1 — Buffer overflow in `hashmap_set` (short key, Invalid write)

```
==79748== Invalid write of size 1
==79748==    at 0x4858FE4: strcpy (vg_replace_strmem.c:565)
==79748==    by 0x400151A: hashmap_set (hashmap_test.c:125)
==79748==    by 0x4001796: main (hashmap_test.c:180)
==79748== Address 0x4ab1308 is 0 bytes after a block of size 8 alloc'd
==79748==    at 0x484F8A8: malloc (vg_replace_malloc.c:446)
==79748==    by 0x4001500: hashmap_set (hashmap_test.c:124)
==79748==    by 0x4001796: main (hashmap_test.c:180)
```

On line 124, `entry->key = malloc(sizeof(key))` allocates only 8 bytes which is the size of a `char*` pointer on a 64-bit system, instead of `strlen(key) + 1` bytes needed to store the string. When `strcpy` copies the key into this undersized buffer it writes past the end of the allocation. This is a heap buffer overflow that could corrupt adjacent memory or be exploited by an attacker supplying a long key. Both Infer and clang-analyzer flagged this in Exercise 1.

### Screenshot 2 — Buffer overflow in `hashmap_set` (long key, Invalid write)

```
==79748== Invalid write of size 1
==79748==    at 0x4858FF6: strcpy (vg_replace_strmem.c:565)
==79748==    by 0x400151A: hashmap_set (hashmap_test.c:125)
==79748==    by 0x4001796: main (hashmap_test.c:180)
==79748== Address 0x4ab1326 is 22 bytes after a block of size 16 in arena "client"
```

Because `entry->key` was written out of bounds during `hashmap_set`, reading it back via `strcmp` in `hashmap_get` also reads beyond the allocated block. This is a direct consequence of the buffer overflow in Screenshot 1. A single bug causing both a write and a read violation. Valgrind tracks both independently, making it easy to trace the root cause.

### Screenshot 3 — Invalid read in `hashmap_get` (consequence of overflow)

```
==79748== Invalid read of size 1
==79748==    at 0x485A2A0: strcmp (vg_replace_strmem.c:941)
==79748==    by 0x4001674: hashmap_get (hashmap_test.c:154)
==79748==    by 0x40017A3: main (hashmap_test.c:182)
==79748== Address 0x4ab1308 is 0 bytes after a block of size 8 alloc'd
==79748==    at 0x484F8A8: malloc (vg_replace_malloc.c:446)
==79748==    by 0x4001500: hashmap_set (hashmap_test.c:124)
==79748==    by 0x4001796: main (hashmap_test.c:180)
```

When deleting the last entry in a hash bucket, `field->size` becomes 0 and `realloc(ptr, 0)` is called. The C standard defines this behavior as implementation-defined, meaning it may return `NULL` or a unique non-null pointer depending on the platform. This makes the code non-portable and potentially unsafe. Neither Infer nor clang-analyzer detected this issue in Exercise 1.

### Screenshot 4 — `realloc()` with size 0

```
==79748== realloc() with size 0
==79748==    at 0x4856F4F: realloc (vg_replace_malloc.c:1801)
==79748==    by 0x40015EA: hashmap_set (hashmap_test.c:141)
==79748==    by 0x4001809: main (hashmap_test.c:194)
==79748== Address 0x4ab1150 is 0 bytes inside a block of size 24 alloc'd
==79748==    at 0x484F8A8: malloc (vg_replace_malloc.c:446)
==79748==    by 0x40014B7: hashmap_set (hashmap_test.c:118)
==79748==    by 0x4001731: main (hashmap_test.c:169)
```

The function `hashmap_get` internally allocates a copy of the value using `malloc` and returns it to the caller. However, the API does not clearly indicate that the caller is responsible for freeing this memory, which makes the ownership model unclear. In the original `main()`, `ret` is never freed, causing a definite memory leak. Infer detected a similar pattern in Exercise 1, and Valgrind confirms it at runtime by pointing to the exact allocation site.

### Screenshot 5 — Memory leak in `hashmap_get` and error summary

```
==79748== 4 bytes in 1 blocks are definitely lost in loss record 1 of 1
==79748==    at 0x484F8A8: malloc (vg_replace_malloc.c:446)
==79748==    by 0x4001686: hashmap_get (hashmap_test.c:155)
==79748==    by 0x400173E: main (hashmap_test.c:171)
==79748==
==79748== LEAK SUMMARY:
==79748==    definitely lost: 4 bytes in 1 blocks
==79748==    indirectly lost: 0 bytes in 0 blocks
==79748==    possibly lost: 0 bytes in 0 blocks
==79748==    still reachable: 0 bytes in 0 blocks
==79748==    suppressed: 0 bytes in 0 blocks
==79748==
==79748== For lists of detected and suppressed errors, rerun with: -s
==79748== ERROR SUMMARY: 64 errors from 5 contexts (suppressed: 0 from 0)
```

The LEAK SUMMARY confirms a definite memory leak of 4 bytes originating inside `hashmap_get` at line 155, where a `malloc`-allocated copy of the value is returned to the caller without any indication of ownership. The original `main()` function never frees the returned pointer, leaving the allocation unreachable at program exit. The ERROR SUMMARY reports 64 errors across 5 contexts, illustrating how a small number of root cause bugs can expand into many individual violations. In this case, the single `sizeof(key)` mistake in `hashmap_set` accounts for most of the errors, as it causes repeated invalid writes and reads during key operations. Infer detected a similar ownership issue in Exercise 1, and Valgrind confirms it at runtime by identifying the exact allocation site and call stack.

### Conclusion:

Valgrind found the buffer overflow caused by `strcpy` and the portability issue with `realloc(0)` that Infer and clang-analyzer missed in Exercise 1, while both static analyzers detected the use-after-free in `hashmap_new` that Valgrind missed entirely because the failure path was never executed. This demonstrates that dynamic and static analysis are complementary rather than interchangeable.